CI-DEMO-PYTHON · CONTINUOUS INTEGRATION DEMO

# Demo Script

*A condensed, copy-paste-ready run sheet for the live walkthrough — same content as docs/WALKTHROUGH.md, formatted for scanning quickly while presenting or following along. About 10 minutes end to end.*

A condensed, copy-paste-ready run sheet for the live walkthrough — same content as docs/WALKTHROUGH.md, formatted for scanning quickly while presenting or following along. About 10 minutes end to end.

* **On a corporate network?** If GitHub is blocked, see Corporate network access at the very end of this document before starting.

## Before starting

- **Fork the repo, then clone your fork** — not this one. Cloning `alfredorueda/ci-demo-python` directly won't let you push branches or open pull requests, since you don't have write access to it:
  1. On GitHub, go to `https://github.com/alfredorueda/ci-demo-python` and click **Fork** (top-right).
  2. Clone *your* fork, replacing `<your-username>`:

     COPY THIS BLOCK

     ```
     git clone https://github.com/<your-username>/ci-demo-python.git
     cd ci-demo-python
     ```

  Every command below runs inside your fork.
- **Install dependencies** — `pytest` is not installed by default, it comes from `requirements.txt`:

  COPY THIS BLOCK

  ```
  python3 -m venv .venv
  source .venv/bin/activate   # Windows: .venv\Scripts\Activate.ps1
  pip install -r requirements.txt
  ```

  Confirmed it worked if the terminal prompt now starts with `(.venv)` and `pytest --version` prints a version instead of "command not found".
- On `main`, terminal open, the repository's **Actions** tab open in a browser tab.
- Confirm the baseline:

COPY THIS BLOCK

```
git checkout main
git pull
pytest -v
```

Expected: `15 passed`.

## 1. Baseline (~1 min)

COPY THIS BLOCK

```
pytest -v
```

`15 passed` in ~0.02s — no Docker, no database, no network call. That's what makes the rest of this run fast.

## 2. Branch, then break the build on purpose (~2 min)

> ⚠️ **STOP — switch branch before editing or committing anything.** `main` is protected: no direct pushes, no exceptions, not even for repository admins. Run the command below *first*, before touching `portfolio_domain/portfolio.py`. If you forget and commit on `main` by mistake, nothing is lost — see "Committed straight to `main` by mistake?" under "If something stalls" — but saving yourself that detour starts here:

COPY THIS BLOCK

```
git checkout -b break-the-build
```

In `portfolio_domain/portfolio.py`, inside `buy(...)`, comment out:

COPY THIS BLOCK

```
self._cash = self._cash.subtract(cost)
```

COPY THIS BLOCK

```
pytest -v
```

Expected: `2 failed, 13 passed` — `test_buy_deducts_cash_balance` and `test_sell_increases_cash_balance`. One bug, two broken behaviors: this is why a suite covers more than one scenario.

## 3. Push and open a pull request (~2 min)

COPY THIS BLOCK

```
git add -A
git commit -m "Introduce a bug on purpose for the CI demo"
git push -u origin break-the-build
gh pr create --fill
```

(No `gh`? Use the "Compare & pull request" banner GitHub shows on the repository page after the push.)

## 4. Watch the check fail (~2 min)

- Refresh the PR page: the check goes yellow (queued/running) → red, in about 10–20 seconds.
- **Details** on the failed check → same `pytest` output as local.
- **Merge** button: greyed out — "*Merging is blocked — Required statuses must pass before merging.*" Applies to everyone, including repository admins — that's the branch protection rule on `main`.

## 5. Fix it and watch it go green (~2 min)

Uncomment the line:

COPY THIS BLOCK

```
self._cash = self._cash.subtract(cost)
```

COPY THIS BLOCK

```
pytest -v
git add -A
git commit -m "Fix the cash balance bug"
git push
```

Refresh the PR: check turns green → **Merge** button active → merge (`Squash and merge` is a fine default).

## 6. Recap (~1–2 min)

- This whole loop took minutes only because the suite is fast and infrastructure-free — a slow one trains people to stop waiting for it, which is how bugs like this reach `main` on real projects.
- Branch protection turns "please run the tests before merging" into "you cannot merge until they pass" — enforced, not requested.
- The entire pipeline is `.github/workflows/ci.yml` — versioned and reviewed like any other code.

## If something stalls

- Actions runs typically take 10–20s to go from "queued" to a result — that gap is normal, not a failure.
- Branch protection can be turned off from **Settings** → **Branches** if `main` needs to be unblocked for any reason.
- **Committed straight to `main` by mistake** (forgot to branch first, under time pressure — it happens to everyone)? Nothing is lost: branch protection blocked the *push*, so the commit only ever existed locally. Move it to a branch instead of losing it:

  COPY THIS BLOCK

  ```
  git branch break-the-build      # snapshot the commit onto a new branch
  git reset --hard origin/main    # bring local main back in sync
  git checkout break-the-build    # keep working from here
  ```

  Then continue from step 3 (push the branch, open a PR) as normal. This is the same protection at work as the rest of this document, just catching a slip instead of a deliberate bad change — that's the point: it doesn't ask why the push happened, it just requires a PR either way.
- `pytest: command not found`? The virtual environment isn't active — re-run `source .venv/bin/activate` (macOS/Linux) or `.venv\Scripts\Activate.ps1` (Windows) from the "Install dependencies" step above.

---

For every step explained in full, with diagrams, see docs/WALKTHROUGH.md. To repeat this same exercise on your own fork afterwards, see its "Try it yourself" section.

---

## Corporate network access

If GitHub is blocked on your corporate network, `git clone`, `git push`, and friends may fail with a connection or timeout error. This is a network-level restriction, unrelated to this repository or this walkthrough — check with your organization: many companies already document the exact proxy configuration needed for `git`/GitHub in an internal channel (for example, shared over Teams). Ask your IT/network team or your training organizer if you're not sure where to find it.